# Robby take-home: follow-up engine

Hey! Thanks for making it this far. This take-home mirrors a real thing we've built: helping techs and owners follow up on open quotes before the customer buys from someone else.

**Time budget: 1 hour max.** We mean it; we'd rather see a smaller thing done thoughtfully than a big thing done sloppily. Note in your README where you stopped and what you'd do next; that counts as part of the submission. AI coding tools are totally fine (we use them heavily) - **just make sure to submit all conversation history along with the rest of your deliverables.** (For instructions on how to do this with Claude Code, see the /export command here).

## The problem

Home-service techs create quotes for homeowners. Quotes sit open for days or weeks. Homeowners open (view) their quotes, sometimes reply, sometimes ghost. Right now, following up is manual: the owner scrolls a list of ~60 open quotes every morning and texts whoever he remembers. Deals die because nobody followed up at the right time.

You're building the MVP of an **automated follow-up engine** that decides *who* to contact, *when*, drafts the message, and (after approval) "sends" it.

## What we give you

A `seed/` folder (attached zip) with two JSON files:

- `quotes.json`: ~30 quotes: `{id, customer_name, customer_phone, tech_name, amount, status (open|accepted|dismissed), created_at, last_contact_at}`
- `events.jsonl`: a stream of timestamped events: `quote_viewed`, `customer_replied`, `message_sent`, `quote_accepted`. Some events are duplicated and some arrive out of order - that's intentional to imitate real webhook streams.

There's no real SMS/email involved. "Sending" can mean writing to an outbox (a table, a file, a log - your call) - but design it as if a real integration sits behind your interface, and note how it would work in production.

## What to build

1. **Ingest** the events into whatever storage you like
2. **A follow-up policy** that, for a given "now", produces a prioritized list of follow-up candidates with a *reason*, e.g.:
    - customer viewed the quote recently but nobody has responded
    - big quote with no contact in N days

    - document any other signals you would look for
3. **Message drafting**
4. **A human in the loop draft → approve → send flow**: drafts are created, a human approves, then the send happens. Guardrails we care about:
    - never message the same customer twice within a cooldown window
    - never message about an accepted or dismissed quote
    - re-running your engine (or retrying a failed send) must never double-send
5. **A way to run / display your work.** Localhost is fine.

## What to include in your deliverables

- The code
- Your coding agent conversation(s)
- Tests for the parts you think deserve tests (we do *not* want 100% coverage, we just want to see what you consider risky)
- A README with:
    - how to run it
    - your follow-up policy and why
    - **~2 paragraphs, no code:** what changes when this runs for a parent company with 50 shops, each with their own quotes, phone numbers, and opinions about what "follow up" means?
    - what you'd build next with another day

## What we're evaluating

Correctness under messy input, the safety guardrails, how you model quote/message state, and the judgment calls in your README. Not: UI, deployment, auth, or framework choice.

You must be able to defend everything you submit. **Use of AI is not an excuse to outsource judgement and reasoning**. If you're using AI for your write ups, make sure you've read it all first before submitting.

Questions welcome; asking a sharp clarifying question is a good sign, not a bad one.